Test Plan Template
(IEEE 829-1998 Format)

Test Plan Identifier

Introduction

This document provides a **comprehensive testing strategy** for the **CyberID: Cybersecurity Intrusion Detection System (IDS)**. The system is designed to detect network intrusions using **machine learning models (SVM)** and provides a **web-based dashboard** for administrators to analyze threats.
Objectives:

- Ensure **accurate threat detection** (normal vs. abnormal traffic).
- Validate **file upload, processing, and visualization** workflows.
- Confirm **real-time dashboard updates** with statistical insights.
- Test **security, performance, and usability** under expected conditions.

Test Items

ML Model (Python, sklearn)

- Built the model using sklearn's SVM model
- Utilized MinMaxScaler to scale the inputs to a standard range
- Downloaded the trained model and saved it as a .pkl file

Frontend (HTML/CSS/JavaScript)

- Dashboard UI:
  - Correct rendering of charts (Pie, Bar, Stacked Bar).
  - Dynamic threat count updates.
  - Responsive design (mobile, tablet, desktop).
- File Upload:
  - Drag-and-drop functionality.
  - File validation (CSV, log files).
  - Error handling for invalid files.
- Terminal Simulation:
  - Real-time log processing messages**.**

Backend (Python/Flask)

- API Endpoints:
  - /analyze – Accepts file uploads, processes logs, triggers ML prediction.
  - /results – Displays threat analysis in tabular format.
  - /api/dashboard-data – Provides JSON data for charts.
  - /api/analyze-log – parse data and transfer it across pages
- Session Management:
  - Persists uploaded filename across requests.
- ML Model Integration:
  - Predicts normal/abnormal logs.
  - Generates explanations for flagged threats.

Database & Data Processing

- CSV Parsing:
  - Handles standard network log formats.
- Feature Scaling:
  - Applies MinMaxScaler for numerical features.
- Statistical Aggregation:
  - Computes attack distribution, failed logins, protocol usage.

Features To Be Tested

1. **Functional Testing**

   a. **File Upload & Processing**

Test Case	Expected Outcome
Upload valid CSV (correct format)	File processes, results display in dashboard.
Upload non-CSV file (e.g., .txt)	Error message: "Invalid file type."
Empty file upload	Error message: "No data detected."
Large file (>10MB)	Processes without timeout (performance check).

   b. **Dashboard Functionality**

Test Case	Expected Outcome
Load dashboard	Charts render without errors.
Filter abnormal logs	Only "abnormal" entries display.
Hover on chart tooltips	Displays exact percentages.
Refresh dashboard	Data updates without page reload.

c. **API Endpoints**

Test Case	Expected Outcome
POST /analyze (valid file)	Returns 200 OK, stores file in session.
GET /results (no file)	Redirects to homepage.
GET /api/dashboard-data	Returns JSON with attack_type_stats, service_stats, etc.

d. **Machine Learning Model**

Test Case	Expected Outcome
Predict normal traffic	Returns "prediction": "normal".
Predict abnormal traffic	Returns "prediction": "abnormal" + explanations.
Malformed input data	Handles gracefully (no server crash).

2. Non-Functional Testing

a. **Performance Testing**

Test Case	Expected Outcome
100 concurrent file uploads	Avg. response time < 2s.
Dashboard load time	Charts render in < 1s.

b. **Security Testing**

Test Case	Expected Outcome
Upload malicious file (e.g., .exe)	Rejected with "Invalid file type."
Session hijacking attempt	Invalidates tampered session.

c. **Usability Testing**

Test Case	Expected Outcome
Mobile view	Charts resize correctly.
Error messages	Clear, actionable guidance.

Features Not To Be Tested

- Third-party APIs (none integrated).
- Model retraining (assumes pre-trained model correctness).
- Browser compatibility (limited to Chrome).

Approach

1. **Manual Testing**
   - **UI/UX:** Verify button clicks, form submissions, error handling.
   - **Edge Cases:** Empty files, corrupted CSVs, session timeouts.
2. **Automated Testing (Optional)**
   - **API Tests:** Postman for /analyze, /results, /api/analyze-log, /api/dashboard-data
   - **Regression Tests:** Selenium for dashboard workflows.
3. **Test Data**
   - **Sample Logs:**
     - logfile.csv (test log file with log entries)

Item Pass/Fail Criteria

Component	Pass Criteria	Fail Criteria
File Upload	Accepts CSV, stores in session.	Crashes on valid files.
Dashboard	Charts load, no JS errors.	Blank charts or stuck loading.
API(/analyze)	Returns 200 OK, processes file.	Returns 500 Internal Error.
ML Model	Consistent predictions.	Random normal/abnormal flips.

Suspension Criteria and Resumption Requirements

- **Suspend if:**
  - Core feature (file upload) fails in >3 test cases.
  - Dashboard crashes on multiple browsers.
- **Resume after:**
  - Defects fixed, regression tests pass.

Test Deliverables

- System test plan
- System test cases
- System test scripts
- Traceability matrix between requirements and users

Test Tasks

1. **Environment Setup**
   - Install Python 3.8+, Flask, pandas, scikit-learn.
   - Configure test CSV files.
2. **Execution**
   - Run test cases, log defects.
3. **Retesting**
   - Verify fixes, update status.

Environmental Needs

1. Hardware Requirements

Development & Testing Environment

- **Primary Workstation:**
  - **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7 (4+ cores, 2.5GHz+)
  - **RAM:** 8GB minimum (16GB recommended for ML processing)
  - **Storage:** 256GB SSD (for faster file processing)
  - **GPU (Optional):** NVIDIA GTX 1060 or higher (for accelerated ML inference)
- **Server (If Deployed Locally):**
  - **Processor:** Intel Xeon or AMD EPYC (8+ cores)
  - **RAM:** 16GB+ (for handling concurrent requests)
  - **Storage:** 500GB+ SSD (for log storage and database operations)
- **Networking:**
  - **Internet Connection:** Stable broadband (10Mbps+ for testing API responses)
  - **Local Network:** Gigabit Ethernet/Wi-Fi 5 (for internal client-server testing)

Specialized Hardware (If Applicable)

- **Security Testing Tools:**
  - **Traffic Simulators:** iPerf, Ostinato (for network load testing)
  - **Packet Analyzers:** Wireshark, tcpdump (for log validation)

2. Software Requirements

Backend Technologies

Component	Version	Purpose
Python	3.8+	Core programming language
Flask	2.0+	Web framework for API endpoints
Werkzeug	2.0+	WSGI utility library
Jinja2	3.0+	Templating engine for HTML rendering
Pandas	1.3+	Data processing for log analysis
Scikit-learn	1.0+	Machine learning (SVM, Random Forest)
Joblib	1.1+	Model persistence (loading .pkl files)
MinMaxScaler	N/A	Feature normalization

Database & Storage

Component	Version	Purpose
CSV Files	N/A	Primary log storage format

Frontend Technologies

Component	Version	Purpose
HTML5	-	Dashboard structure
CSS3	-	Styling and responsive design
JavaScript (ES6+)	-	Dynamic chart rendering (Chart.js)
Chart.js	3.7+	Visualization for attack statistics

Operating System Compatibility

OS	Version	Notes
Windows	10, 11	Primary development/testing OS
macOS	12.0+ (Monterey)	For cross-platform validation
Linux	Ubuntu 20.04+/CentOS 8+	Server deployment

Networking & Security

Component	Purpose
HTTPS (TLS 1.2+)	Secure API communication
CORS Policies	Restrict unauthorized frontend access
Session Encryption	Flask SECRET_KEY for session security

Testing & Debugging Tools

Tool	Purpose
Postman	API endpoint validation
Browser DevTools	Debug frontend rendering issues
Jupyter Notebook	ML model validation (optional)

Additional Dependencies

- **Virtual Environment:** venv or conda (for dependency isolation)
- **Version Control:** Git (GitHub/GitLab)
- **CI/CD (Optional):** GitHub Actions (for automated testing)

Responsibilities

1. System test lead is responsible for System Test Plan
2. System Test Plan addresses hardware required, software required, testers required for testing the project. Also gives the System Test Schedule which gives start date and end date of the system testing
3. In the case of SCRUM Methodology, the test team provides the Test Schedule on Sprint plan
4. Test group writes the test cases by understanding the requirements from the SRS Document

Role	Tasks
Test Lead	Approve test plan, manage execution.
Test Engineers	Execute cases, log defects.
Developers	Fix priority bugs.

Staffing and Training Needs

1. Testers will undergo **domain-specific training** on web applications, ML and DL.
2. **Team Composition:**
   - **1 Test Lead** – Responsible for test strategy and oversight.
   - **2 Test Engineers** – Execute test cases and analyze results.

Schedule

Phase	Start	End
Setup	Apr 4	Apr 10
Functional Tests	Apr 11	Apr 15
Bug Fixing	Apr 16	Apr 19
Final Sign-off	May 20	May 20

Risks and Contingencies

1. If engineers or key team members resign or become unavailable due to unforeseen circumstances, the project schedule and test execution will be impacted.
2. Lack of proper test data could lead to incomplete testing and potential defects in production.
3. Failing to meet security standards or compliance requirements may require additional testing and fixes.
4. Unexpected performance issues under load testing may require optimization and additional development time.
5. Delays in feedback from end-users or stakeholders during User Acceptance Testing (UAT) may impact final release schedules.

Approvals

The following people wil approve the test plan:

- Customer
- Developers
- Testers
- DevOps Engineers
- Project Manager
- Scrum Master
- Senior Management Team

Status: ☑ Approved